

URSA: The Universal Research and Scientific Agent

Michael Grosskopf¹, Nathan DeBardleben¹, Russell Bent¹, Rahul Somasundaram¹, Isaac Michaud¹, Arthur Lui¹, Alexius Wadell¹, Warren D. Graham¹, Golo A Wimmer¹, Sachin Shivakumar¹, Joan Vendrell Gallart¹, Harsha Nagarajan¹, Jamal Mohd-Yusof¹, Adela Habib¹, and Earl Lawrence¹

¹ Los Alamos National Laboratory, Los Alamos, New Mexico, United States

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Chris Vernon](#) 

Reviewers:

- [@rahuldevikar](#)
- [@vbcherepanov](#)

Submitted: 04 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Large language models (LLMs) (Zhao et al., 2026) are increasingly being integrated within agentic workflows to automate complex tasks, including software engineering, information retrieval, and scientific research. These developments have created new opportunities for AI systems that assist researchers with planning, computation, analysis, and validation. Here, we present URSA (Universal Research and Scientific Agent), an open-source framework for building modular and extensible AI agents specifically for scientific workflows. URSA enables researchers to compose domain-specific agents by integrating large language models with the appropriate scientific computational tools and knowledge, supporting both general-purpose scientific reasoning and specialized research applications.

Statement of need

Scientific research increasingly requires AI systems that can integrate reasoning with external software, simulation codes, computational resources, and specialized literature (Boiko et al., 2023). While existing agentic frameworks provide general-purpose planning and tool use, researchers often need specialized agents tailored to particular research questions.

URSA addresses this need through a collection of reusable core agents that implement common research capabilities such as hypothesis generation, planning, and execution. Furthermore, building on these core agents, URSA includes several specialized agents that demonstrate how the framework can be extended to research applications by integrating scientific computational tools and domain knowledge. These agents support tasks such as molecular dynamics simulations, large-scale numerical simulations, and optimization. Beyond serving as end-user applications, they also act as reference implementations that illustrate how new scientific agents can be developed by composing URSA's core reusable components. Importantly, URSA further provides multiple ways to deploy and orchestrate these agents. Researchers can interact with the framework through a command-line interface, web dashboard, Model Context Protocol (MCP), or Python API, while execution environments such as Agent Teams and Agent Symposia enable agents to collaborate or independently evaluate scientific problems.

State of the field

General-purpose agentic systems such as Claude Code (Anthropic, 2026) and Codex (OpenAI, 2026) have demonstrated the effectiveness of LLMs for software engineering tasks, including code generation and debugging. These systems excel at programming assistance but are not designed as extensible platforms for developing scientific agents. In parallel, several systems

have also been proposed specifically for scientific research, including Sakana AI's AI Scientist (Lu et al., 2024), Google's Co-Scientist (Gottweis et al., 2026), SciAgents (Ghafariollahi & Buehler, 2025), Agent Laboratory (Schmidgall et al., 2025), ChemCrow (Bran et al., 2024), and OpenAI's Deep Research (OpenAI, 2025). While these systems demonstrate the growing potential of AI-assisted research, many are designed as end-to-end research assistants.

In contrast, URSA is an open-source software framework for constructing scientific agents rather than a single predefined assistant. It provides reusable core agents for common research capabilities, example implementations that demonstrate how these components can be extended to specialized applications, multiple user interfaces for deploying workflows, and execution environments such as Agent Symposia that support collaborative and deliberative multi-agent reasoning. Together, these capabilities enable researchers to build, deploy, and orchestrate customized AI-assisted workflows across scientific disciplines.

Software design

URSA is built on top of LangGraph (LangChain Inc., 2026) and is agnostic to the underlying LLM. Its software architecture is organized around three main concepts: (i) reusable core and domain-specific agents, (ii) multiple user interfaces that expose a common execution engine, and (iii) execution environments that orchestrate collaboration among multiple agents.

Agent Framework

The separation of the framework into (i) core and (ii) domain-specific agents enables the same architectural components to be reused across different research problems.

Core Agents

URSA's core agents include, but are not limited to, the following:

- **Planning Agent:** This agent decomposes a user-specified scientific problem into a sequence of executable tasks. Implemented as a LangGraph workflow, it consists of three LLM-driven nodes: a planner node that generates an initial research plan, a reviewer node that evaluates and iteratively refines the plan, and a formalizer node that converts the approved plan into a structured JSON representation.
- **Execution Agent:** This agent carries out research tasks specified either in natural language or in the structured JSON format produced by the Planning Agent. It interacts with tools through LangGraph tool calls and through the Model Context Protocol (MCP), allowing virtually any user-provided executable to be incorporated into agent workflows. The Execution Agent also includes built-in tools for code generation and execution, file reading and writing, and system command execution. To improve safety, proposed system commands are screened by an LLM-driven safety node before execution.

Domain-Specific Agents

- **Simulation Agent:** The Simulation Agent supports the use of computationally intensive numerical simulation codes. This agent is constructed by orchestrating multiple instances of the core Execution Agent within a LangGraph workflow. One execution agent is responsible for documentation and knowledge acquisition, while another is responsible for simulation setup, execution, debugging, and analysis. The documentation stage gathers information from user-provided manuals, web resources, scientific literature, and RAG-based knowledge bases (Gao et al., 2023) to construct a task-specific user guide. This guide is then passed to the simulation stage, which uses it to configure and execute simulation campaigns, analyze outputs, and iteratively resolve execution failures.

- Optimization Agent: The Optimization Agent is a self-contained LangGraph workflow for formulating and solving optimization and inverse-design problems from natural-language input. The agent first extracts the optimization problem, converts it into a structured mathematical representation, and optionally discretizes the problem when infinite-dimensional variables or constraints are detected. It then selects an appropriate solver, generates executable optimization code, runs feasibility checks using a dedicated tool, verifies the resulting formulation, and produces a final explanation of the solution.
- LAMMPS Agent: To illustrate how specialized a URSA agent can be, the Large-scale Atomic/Molecular Massively Parallel Simulator (LAMMPS) (Thompson et al., 2022) Agent provides a dedicated workflow for molecular dynamics calculations (Somasundaram et al., 2026). The agent is capable of autonomously orchestrating the full lifecycle of a molecular dynamics simulation, including interatomic potential selection, generation of LAMMPS input scripts, simulation execution, iterative error recovery, and post-processing of results. A key feature of the agent is its ability to leverage other agents within the URSA ecosystem for tasks such as visualization, literature review, and validation against published results.

User Interfaces

URSA's framework is exposed through multiple interfaces that share the same underlying engine.

Command-Line Interface

The command-line interface (CLI) provides an interactive Read-Eval-Print Loop (REPL) for executing agents and composing scientific workflows. A typical session is launched using

```
ursa --config config.yaml
```

where the YAML configuration specifies details such as the desired LLM and other runtime options. Within the REPL, users can invoke individual agents using commands such as

```
ursa> plan Write a workflow for computing elastic constants  
        using a molecular dynamics simulation.  
ursa> execute Execute the plan.
```

Web Dashboard

URSA also provides a browser-based dashboard that exposes the same agent capabilities through a graphical interface. After installing the optional dashboard dependencies, the server can be started with

```
ursa-dashboard
```

or configured explicitly, for example,

```
ursa-dashboard --host 127.0.0.1 --port 8080
```

Python Interface

For integration into scientific software, URSA can be imported directly as a Python package. Agents may be instantiated, configured, and executed within Python scripts or Jupyter notebooks. A typical workflow consists of creating an agent instance and invoking it with a natural-language task:

```
from langchain.chat_models import init_chat_model  
from langchain_core.messages import HumanMessage  
from ursa.agents import ExecutionAgent
```

```

llm = init_chat_model(model="openai:gpt-5.4")
agent = ExecutionAgent(llm=llm)

result = agent.invoke({
    "messages": [
        HumanMessage(
            content="Write and run a Python script that prints the first 10 prime numbers."
        )
    ],
    "workspace": "./ursa-script-workspace",
})
print(result["messages"][-1].content)

```

118 The Python interface is particularly useful for integration with existing simulation codes and
 119 high-performance computing workflows.

120 Agent Execution Environments

121 URSA provides execution environments for composing multiple agents into larger scientific
 122 workflows.

123 Agent Teams

124 The Agent Teams environment coordinates multiple agents through a hierarchical collaboration
 125 model. A designated principal investigator (PI) agent receives the user's request, decomposes
 126 it into subtasks, and delegates those tasks to specialized agents through focused prompts.
 127 URSA's core Execution Agent serves as the default PI because it supports the delegation tools
 128 required to coordinate the team, although users may substitute their own tool-enabled agents.
 129 URSA's documentation includes several examples of Agent Teams, including workflows in
 130 which a PI delegates literature review tasks to a Chat Agent before synthesizing the results
 131 into a final response.

132 Agent Symposia

133 The Agent Symposia environment follows a peer-review collaboration model, rather than a
 134 hierarchical one. Each participant agent develops its own solution or recommendation, critiques
 135 the responses of other participants, and iteratively refines its reasoning. The final response
 136 is then synthesized from the discussion. Different participants may bring different tools,
 137 assumptions, models, and reasoning styles. This environment is particularly well suited for
 138 scientific tasks that benefit from multiple independent perspectives, such as research planning,
 139 hypothesis generation, or evaluation of competing approaches.

140 Research impact statement

141 URSA is actively used in a growing number of scientific research applications, averaging ~2000
 142 downloads a month on PyPI. While the framework was initially developed at Los Alamos
 143 National Laboratory, it is distributed as open-source software and is intended to support
 144 contributions and adoption by the broader scientific community.

145 As one example, [Figure 1](#) shows the use of URSA in the design of inertial confinement
 146 fusion (ICF) capsules. In this workflow, URSA's planning and execution agents were used
 147 to autonomously explore candidate designs and optimize neutron yield. More details of this
 148 application can be found in Grosskopf et al. (2025).

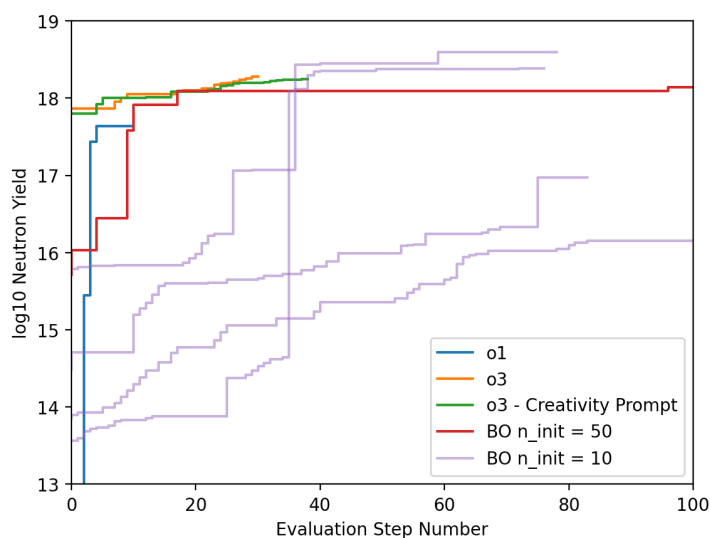


Figure 1: Comparison of URSA to Bayesian optimization (BO) for designing a direct-drive ICF design. The plots show the iterative running maximum neutron yield. Figure taken from Grosskopf et al. (2025).

As a second example, Figure 2 shows atomistic calculations performed by URSA's LAMMPS Agent (Somasundaram et al., 2026) for a high-entropy alloy (George et al., 2019). Such calculations are commonly employed in computational materials science to predict material properties and guide the design of novel materials.

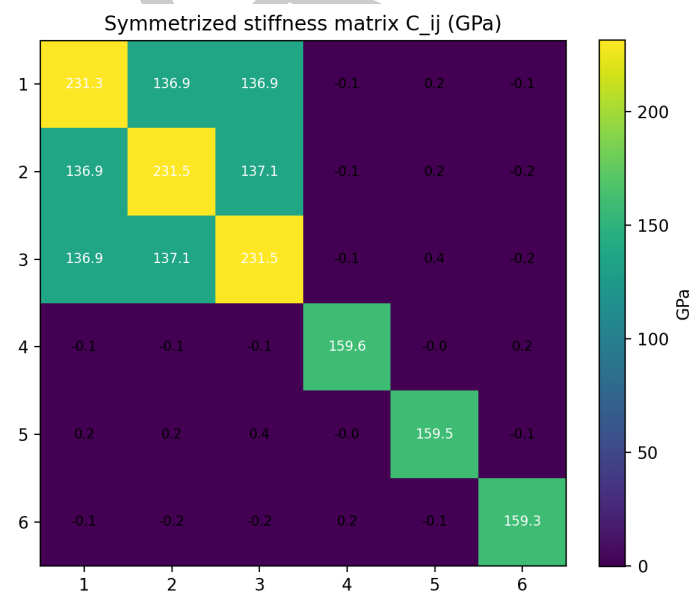


Figure 2: The stiffness tensor, i.e., the elastic constants calculated for the high entropy alloy Co-Cr-Fe-Mn-Ni. The atomistic calculation was performed by the LAMMPS agent.

AI usage disclosure

This work is on AI-driven agentic workflows, so LLMs are invoked at multiple instances within URSA. For example, LLMs were involved in the generation of Figure 1 and Figure 2. During the development of URSA, the framework was also used to assist with implementing specific

portions of its own source code. All such code was thoroughly reviewed and tested by the authors. For the writing of this manuscript, LLMs were used only for minor polishing, such as grammar and spell checks. All LLM-generated text was critically reviewed by the authors for accuracy.

Acknowledgements

This work was supported by the Laboratory Directed Research and Development program of Los Alamos National Laboratory under project number 20250638DI. This research used resources provided by the Los Alamos National Laboratory Institutional Computing Program, which is supported by the U.S. Department of Energy National Nuclear Security Administration under Contract No. 89233218CNA000001. This document has been approved for unlimited release, and was assigned LA-UR-26-27324.

References

- Anthropic. (2026). Claude code. In *GitHub repository*. GitHub. <https://github.com/anthropics/claude-code>
- Boiko, D. A., MacKnight, R., Kline, B., & Gomes, G. (2023). Autonomous chemical research with large language models. *Nature*, 624(7992), 570–578. <https://doi.org/10.1038/s41586-023-06792-0>
- Bran, A. M., Cox, S., Schilter, O., Baldassari, C., White, A. D., & Schwaller, P. (2024). Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5), 525–535. <https://doi.org/10.1038/s42256-024-00832-8>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv Preprint arXiv:2312.10997*.
- George, E. P., Raabe, D., & Ritchie, R. O. (2019). High-entropy alloys. *Nature Reviews Materials*, 4(8), 515–534.
- Ghafarirollahi, A., & Buehler, M. J. (2025). SciAgents: Automating scientific discovery through bioinspired multi-agent intelligent graph reasoning. *Advanced Materials*, 37(22), 2413523.
- Gottweis, J., Weng, W.-H., Daryin, A., Tu, T., Sirkovic, P., Myaskovsky, A., Glowaty, G., Weissenberger, F., Orlandi, A., Popovici, D., & others. (2026). Accelerating scientific discovery with co-scientist. *Nature*, 1–3.
- Grosskopf, M., Debardeleben, N., Bent, R., Somasundaram, R., Michaud, I., Lui, A., Wadell, A., Graham, W. D., Wimmer, G. A., Shivakumar, S., & others. (2025). Ursa: The universal research and scientific agent. *arXiv Preprint arXiv:2506.22653*.
- LangChain Inc. (2026). LangGraph. In *GitHub repository*. GitHub. <https://github.com/langchain-ai/langgraph>
- Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., & Ha, D. (2024). The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv Preprint arXiv:2408.06292*.
- OpenAI. (2025). *Introducing deep research*. <https://openai.com/index/introducing-deep-research/>.
- OpenAI. (2026). Codex. In *GitHub repository*. GitHub. <https://github.com/openai/codex>
- Schmidgall, S., Su, Y., Wang, Z., Sun, X., Wu, J., Yu, X., Liu, J., Moor, M., Liu, Z., & Barsoum, E. (2025). Agent laboratory: Using llm agents as research assistants. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 5977–6043.

- 200 Somasundaram, R., Habib, A., Dang, K., Shivakumar, S., Hill, R. G., Wimmer, G., Mishra, A.,
201 Pachalieva, A., Lui, A., Viswanathan, H., Grosskopf, M., Fensin, S., Bent, R., DeBardeleben,
202 N., & Lawrence, E. (2026). An Agentic Orchestration of Atomistic Simulations. *arXiv*
203 *e-Prints*, arXiv:2607.22596. <https://doi.org/10.48550/arXiv.2607.22596>
- 204 Thompson, A. P., Aktulga, H. M., Berger, R., Bolintineanu, D. S., Brown, W. M., Crozier,
205 P. S., In't Veld, P. J., Kohlmeyer, A., Moore, S. G., Nguyen, T. D., & others. (2022).
206 LAMMPS—a flexible simulation tool for particle-based materials modeling at the atomic,
207 meso, and continuum scales. *Computer Physics Communications*, 271, 108171.
- 208 Zhao, W. X., Zhou, K., Li, J., Tang, T., Dong, Z., Hou, Y., Zhang, B., Min, Y., Zhang, J.,
209 Liu, P., & others. (2026). A survey of large language models. *Frontiers of Computer*
210 *Science*, 20(12), 2012627.

DRAFT